

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT

On

ANALYSIS AND DESIGN OF ALGORITHMS (23CS4PCADA)

Submitted by

Liesha Jajodia (1WA24CS154)

**in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING**



**B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
February to June 2026**

CERTIFICATE

**B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering**



This is to certify that the Lab work entitled “**ANALYSIS AND DESIGN OF ALGORITHMS**” carried out by **Liesha Jajodia (1WA24CS154)** who is bonafide student of **B. M. S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum during the year 2025-26. The Lab report has been approved as it satisfies the academic requirements in respect of Analysis and Design of Algorithms Lab - **(23CS4PCADA)** work prescribed for the said degree.

Pallavi C.V
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Sl. No.	Experiment Title	Page No.
1	Write program to obtain the Topological ordering of vertices in a given digraph.	
2	Implement Johnson Trotter algorithm to generate permutations.	
3	Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.	
4	Sort a given set of N integer elements using Quick Sort technique and compute its time taken.	
5	Sort a given set of N integer elements using Heap Sort technique and compute its time taken.	
6	Implement 0/1 Knapsack problem using dynamic programming. (5 Marks)	
7	Implement All Pair Shortest paths problem using Floyd's algorithm.	
8	(a) Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm. (b) Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.	
9	Implement Fractional Knapsack using Greedy technique.	
10	From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.	
11	Implement "N-Queens Problem" using Backtracking.	

Course outcomes:

CO1	Analyze time complexity of algorithms using asymptotic notations.
CO2	Apply various algorithm design techniques for the given problem.
CO3	Evaluate the appropriate algorithm/design technique for solving a given problem.
CO4	Conduct practical experiments by applying appropriate algorithm design technique to solve problems.

Lab program 1:

Complete program in C

Screenshot of the output to be included for all the cases.

For all sorting programs, graph (plotted in Excel) to be attached and shown for large values of 'n' how time increases.

**NOTE: SIMILARLY INCLUDE FOR ALL THE LAB PROGRAMS (ALL 11).
LEETCODE PROGRAMS ALSO TO BE INCLUDED. Include the Leetcode Programs
which you have executed along with the respective lab program.**

ENSURE THAT EACH LAB PROGRAM STARTS IN A NEW PAGE ONLY.

Lab program 1:

Write program to obtain the Topological ordering of vertices in a given digraph.

SOURCE CODE

```
#include <stdio.h>
#include <stdlib.h>
#define MAX 100

int adj[MAX][MAX];
int indegree[MAX];
int queue[MAX];
int n;

void topologicalSort() {
    int front = 0, rear = -1;
    int count = 0;
    for (int i = 0; i < n; i++)
        indegree[i] = 0;

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (adj[i][j] == 1)
                indegree[j]++;
        }
    }

    for (int i = 0; i < n; i++) {
        if (indegree[i] == 0)
            queue[++rear] = i;
    }

    printf("\nTopological Order: ");

    while (front <= rear) {
        int node = queue[front++];
        printf("%d ", node);
        count++;
        for (int i = 0; i < n; i++) {
            if (adj[node][i] == 1) {
                indegree[i]--;
                if (indegree[i] == 0)
                    queue[++rear] = i;
            }
        }
    }

    if (count != n)
        printf("\nGraph has a cycle! No topological ordering possible.\n");
}
```

```

int main() {
    printf("\nEnter number of vertices: ");
    scanf("%d", &n);

    printf("\nEnter adjacency matrix:\n");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++)
            scanf("%d", &adj[i][j]);
    }

    topologicalSort();
    return 0;
}

```

OUTPUT

```
Enter number of vertices: 4
```

```
Enter adjacency matrix:
```

```
0 1 1 0
```

```
0 0 0 1
```

```
0 0 0 1
```

```
0 0 0 0
```

```
Topological Order: 0 1 2 3
```

Leetcode Problem 207- Course schedule

Lab program 2:

Implement Johnson Trotter algorithm to generate permutations.

SOURCE CODE

```
#include <stdio.h>
int a[10], dir[10];

void print(int n){
    for (int i=0;i<n; i++)
        printf("%d ", a[i]);
    printf("\n");
}

int mobile(int n){
    int m=0,pos=-1;

    for (int i=0; i<n; i++){
        int j=i+dir[i];
        if (j >= 0 && j < n && a[i] > a[j] && a[i] > m) {
            m=a[i];
            pos=i;
        }
    }
    return pos;
}

int main(){
    int n;
    printf("\nEnter number: ");
    scanf("%d", &n);

    for (int i=0;i<n;i++) {
        a[i]=i+1;
        dir[i] = -1;
    }

    print(n);

    while (1) {
        int i = mobile(n);
        if (i == -1) break;
        int j = i + dir[i];
        int t = a[i]; a[i] = a[j]; a[j] = t;
        t = dir[i]; dir[i] = dir[j]; dir[j] = t;
    }
}
```

```
        for (int k = 0; k < n; k++)
            if (a[k] > a[j]) dir[k] *= -1;

        print(n);
    }

    return 0;
}
```

OUTPUT

```
Enter number: 3
1 2 3
1 3 2
3 1 2
3 2 1
2 3 1
2 1 3
```


Lab program 3:

Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.

SOURCE CODE

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int a[5000000],b[5000000];

void merge(int a[],int low,int mid,int high){

    int i=low,j=mid+1,k=low;

    while(i<=mid && j<=high){
        if(a[i]<a[j])
            b[k]=a[i++];
        else
            b[k]=a[j++];
        k++;
    }
    while(i<=mid){
        b[k++]=a[i++];
    }
    while(j<=high){
        b[k++]=a[j++];
    }
    for(int x=low;x<=high;x++)
        a[x]=b[x];
}

void merge_sort(int a[],int low,int high){
    if(low<high){
        int mid=(low+high)/2;
        merge_sort(a,low,mid);
        merge_sort(a,mid+1,high);
        merge(a,low,mid,high);
    }
}
```

```

int main(){
    int n;
    printf("\nEnter number of elements: ");
    scanf("%d",&n);
    printf("\nEnter the elements...");
    for(int i=0;i<n;i++){
        a[i]=rand()%1000;
        printf("%d ",a[i]);
    }
    clock_t start_time=clock();
    merge_sort(a,0,n-1);
    clock_t end_time=clock();
    printf("\nAfter sorting: ");
    for(int i=0;i<n;i++)
        printf("%d ",a[i]);
    float total_time=(float)(end_time-start_time)/CLOCKS_PER_SEC*1000;
    printf("\nTime taken is: %.2f",total_time);
    return 0;
}

```

OUTPUT

```
Enter number of elements: 10
```

```
Entering the elements...41 467 334 500 169 724 478 358 962 464
```

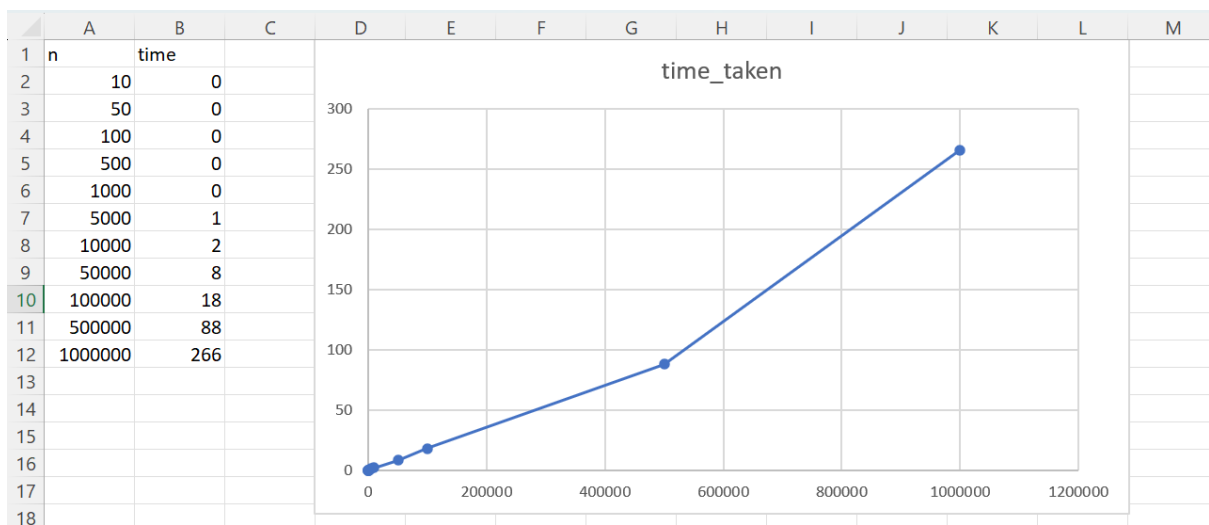
```
After sorting: 41 169 334 358 464 467 478 500 724 962
```

```
Time taken is: 0.00
```

Time taken for each N

N	Time taken
10	0.0
50	0.0
100	0.0
500	0.0
1000	0.0
5000	1.0
10000	2.0
50000	8.0
100000	18.0
500000	88.0
1000000	266.0

Graph:



Lab program 4:

Sort a given set of N integer elements using Quick Sort technique and compute its time taken.

SOURCE CODE

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
int a[5000000];

void swap(int *a,int *b){
    int temp=*a;
    *a=*b;
    *b=temp;
}

int partition(int a[],int low,int high){

    int pivot=a[low];
    int i=low+1;
    int j=high;

    while(i<=j){
        while(i<=high && pivot>=a[i])
            i++;
        while(j>=low && pivot<a[j])
            j--;
        if(i<j)
            swap(&a[i],&a[j]);
    }
    swap(&a[low],&a[j]);
    return j;
}

void quick_sort(int a[],int low,int high){
    if(low<high){
        int j=partition(a,low,high);
        quick_sort(a,low,j-1);
        quick_sort(a,j+1,high);
    }
}
```

```

int main(){

    int n;
    printf("\nEnter number of elements: ");
    scanf("%d",&n);

    printf("\nEnter elements...");
    for(int i=0;i<n;i++){
        a[i]=rand()%1000;
        printf("%d ",a[i]);
    }

    clock_t start=clock();
    quick_sort(a,0,n-1);
    clock_t end=clock();

    printf("\nAfter sorting....");
    for(int i=0;i<n;i++)
        printf("%d ",a[i]);

    float time_taken=((float)(end-start))/CLOCKS_PER_SEC*1000;
    printf("\nTime taken: %.2f",time_taken);

    return 0;
}

```

OUTPUT

```
Enter number of elements: 10
```

```
Entering elements...41 467 334 500 169 724 478 358 962 464
```

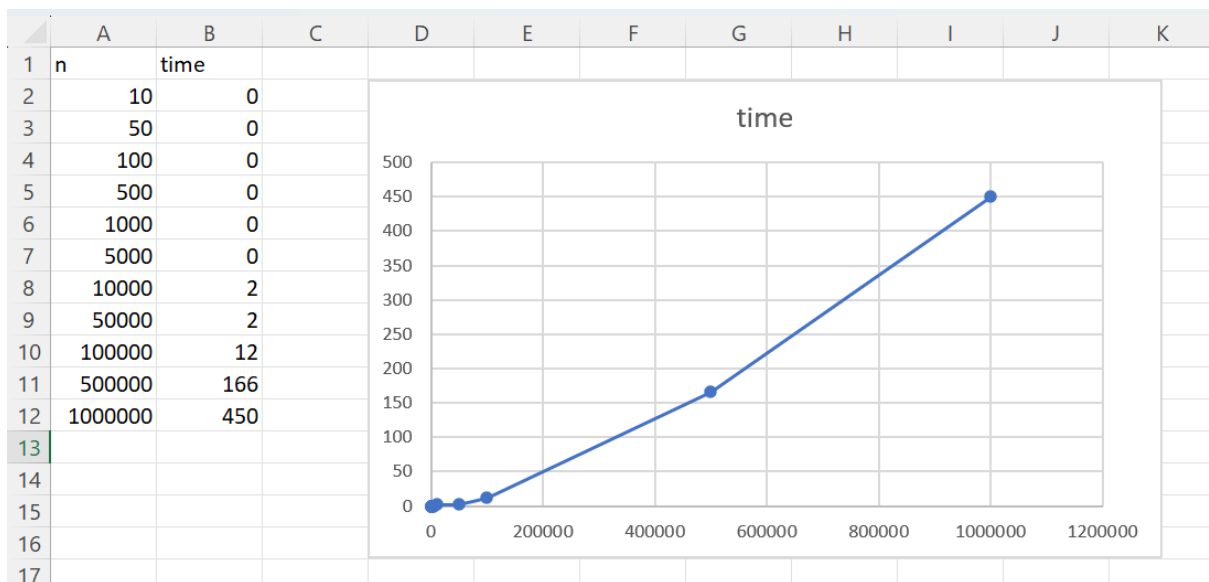
```
After sorting....41 169 334 358 464 467 478 500 724 962
```

```
Time taken: 0.00
```

Time taken for each N

N	Time taken
10	0.0
50	0.0
100	0.0
500	0.0
1000	0.0
5000	0.0
10000	2.0
50000	2.0
100000	12.0
500000	166.0
1000000	450.0

Graph:



Lab program 5:

Sort a given set of N integer elements using Heap Sort technique and compute its time taken.

SOURCE CODE

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int a[5000000];

void heapify(int a[],int i,int n){
    int largest=i;
    int left=2*i+1;
    int right=2*i+2;

    if(left<n && a[left]>a[largest])
        largest=left;
    if(right<n && a[right]>a[largest])
        largest=right;
    if(largest!=i){
        int temp=a[i];
        a[i]=a[largest];
        a[largest]=temp;
        heapify(a,largest,n);
    }
}

void heap_sort(int a[],int n){
    for(int i=(n/2)-1;i>=0;i--){
        heapify(a,i,n);
    }
    for(int i=n-1;i>0;i--){
        int temp=a[0];
        a[0]=a[i];
        a[i]=temp;
        heapify(a,0,i);
    }
}

void print(int a[],int n){
    for(int i=0;i<n;i++){
        printf("%d ",a[i]);
        printf("\n");
    }
}
```

```

int main(){
    int n;
    printf("\nEnter number of elements: ");
    scanf("%d",&n);
    printf("\nEnter elements...");
    for(int i=0;i<n;i++){
        a[i]=rand()%1000;
    }
    print(a,n);
    clock_t start=clock();
    heap_sort(a,n);
    clock_t end=clock();
    printf("\nAfter sorting....");
    print(a,n);
    float time_taken=((float)(end-start))/CLOCKS_PER_SEC*1000;
    printf("\nTime taken: %.2f",time_taken);
    return 0;
}

```

OUTPUT

```

Enter number of elements: 5

Entering elements...41 467 334 500 169

After sorting....41 169 334 467 500

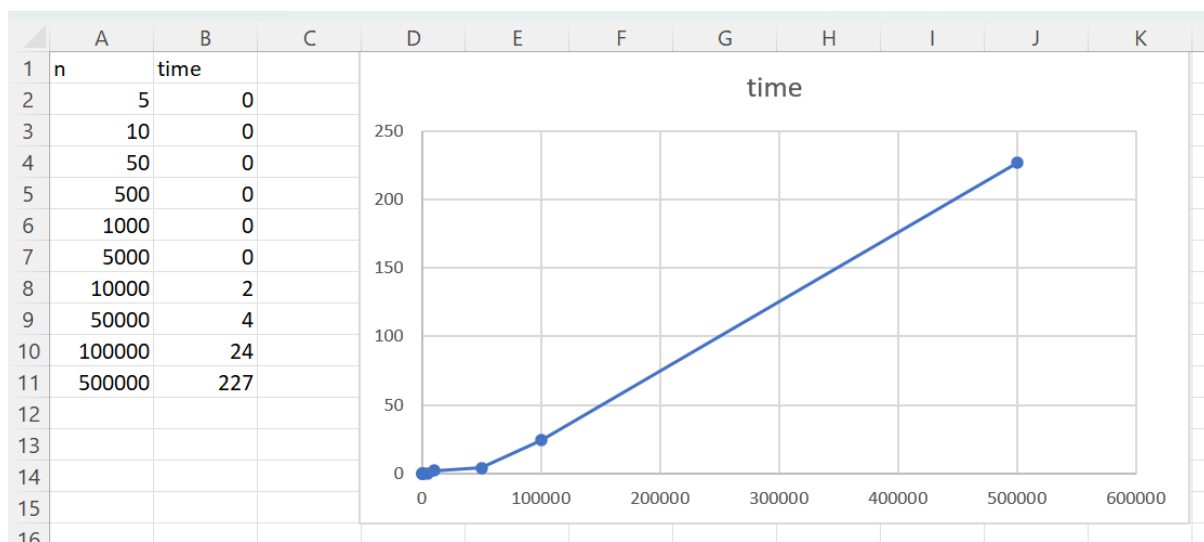
Time taken: 0.00

```


Time taken for each N

N	Time taken
5	0
10	0
50	0
500	0
1000	0
5000	0
10000	2
50000	4
100000	24
500000	227
5	0

Graph:



Lab program 6:

Implement 0/1 Knapsack problem using dynamic programming.

SOURCE CODE

```
#include <stdio.h>

int p[50],w[50],n,m;

void inputs(){
    printf("\nEnter number of items: ");
    scanf("%d",&n);
    printf("\nEnter capacity: ");
    scanf("%d",&m);

    printf("\nEnter weights: ");
    for(int i=1;i<=n;i++)
        scanf("%d",&w[i]);
    printf("\nEnter profits: ");
    for(int i=1;i<=n;i++)
        scanf("%d",&p[i]);
}

int max(int a,int b){
    return (a>b)?a:b;
}

int knapsack(){
    int v[n+1][m+1];
    for(int i=0;i<=n;i++){
        for(int j=0;j<=m;j++){
            if(i==0||j==0)
                v[i][j]=0;
            else if(w[i]>j)
                v[i][j]=v[i-1][j];
            else
                v[i][j]=max(v[i-1][j],(v[i-1][j-w[i]])+p[i]);
        }
    }
    printf("\nOptimal Profit: %d",v[n][m]);
}

int main(){
    inputs();
    knapsack();
    return 0;
}
```

OUTPUT

```
Enter number of items: 4
```

```
Enter capacity: 8
```

```
Enter weights: 3 4 6 5
```

```
Enter profits: 2 3 1 4
```

```
Optimal Profit: 6
```

Lab program 7:

Implement All Pair Shortest paths problem using Floyd's algorithm.

SOURCE CODE

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define INF 99999

void floyd(int n,int R[n][n]){
    for(int k=0;k<n;k++){
        for(int i=0;i<n;i++){
            for(int j=0;j<n;j++){
                if(R[i][j]>(R[i][k]+R[k][j]))
                    R[i][j]=R[i][k]+R[k][j];
            }
        }
    }
}

int main(){
    int n;
    printf("\nEnter number of vertices: ");
    scanf("%d",&n);
    int R[n][n];
    printf("\nEnter adjacency matrix:\n");
    for(int i=0;i<n;i++){
        for(int j=0;j<n;j++){
            scanf("%d",&R[i][j]);
        }
    }
    floyd(n,R);
    printf("\nShortest path:\n");
    for(int i=0;i<n;i++){
        for(int j=0;j<n;j++){
            if(R[i][j]==INF)
                printf("INF ");
            else
                printf("%d ",R[i][j]);
        }
        printf("\n");
    }
    return 0;
}
```

OUTPUT

```
Enter number of vertices: 4
```

```
Enter adjacency matrix:
```

```
0 99999 3 99999
```

```
2 0 99999 99999
```

```
99999 7 0 1
```

```
6 99999 99999 0
```

```
Shortest path:
```

```
0 10 3 4
```

```
2 0 5 6
```

```
7 7 0 1
```

```
6 16 9 0
```

Lab program 8:

(a) Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm.

SOURCE CODE

```
#include <stdio.h>

void prims(int n,int graph[n][n]){

    int visited[n];
    for(int i=0;i<n;i++)
        visited[i]=0;
    int edges=0;
    visited[0]=1;
    int cost=0;
    printf("\nEdge\tWeight\n");

    while(edges<n-1){

        int min=100,x=-1,y=-1;
        for(int i=0;i<n;i++){
            if(visited[i]==1){
                for(int j=0;j<n;j++){
                    if(visited[j]==0 && graph[i][j]!=0){
                        if(graph[i][j]<min){
                            min=graph[i][j];
                            x=i;
                            y=j;
                        }
                    }
                }
            }
        }

        printf("%d - %d\t%d\n",x,y,graph[x][y]);
        cost+=graph[x][y];
        visited[y]=1;
        edges++;
    }
    printf("\nMinimum Cost %d\n",cost);
}
```

```

int main(){
    int n;
    printf("\nEnter number of vertices: ");
    scanf("%d",&n);
    int graph[n][n];
    printf("\nEnter the adjacency matrix:\n");
    for(int i=0;i<n;i++){
        for(int j=0;j<n;j++){
            scanf("%d",&graph[i][j]);
        }
    }
    prims(n,graph);
    return 0;
}

```

OUTPUT

```
Enter number of vertices: 6
```

```
Enter the adjacency matrix:
```

```
0 3 0 0 5 6
```

```
3 0 1 0 4 0
```

```
0 1 0 6 4 0
```

```
0 0 6 0 5 3
```

```
5 4 4 5 0 2
```

```
6 0 0 3 2 0
```

Edge	Weight
------	--------

0 - 1	3
-------	---

1 - 2	1
-------	---

1 - 4	4
-------	---

4 - 5	2
-------	---

5 - 3	3
-------	---

Lab program 8:

(b) Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.

SOURCE CODE

```
#include <stdio.h>

struct Edge{
    int u,v,w;
}e[10],temp;

int parent[10];

int find(int x){
    while(parent[x]!=x)
        x=parent[x];
    return x;
}

int main(){

    int n,m,cost=0;

    printf("\nEnter number of Vertices and Edges: ");
    scanf("%d%d",&n,&m);

    printf("Enter vertex u v and weight w:\n");

    for(int i=0;i<m;i++){
        scanf("%d%d%d",&e[i].u,&e[i].v,&e[i].w);

        for(int j=0;j<m-1;j++){
            for(int k=j+1;k<m;k++){
                if(e[j].w > e[k].w){
                    temp=e[j];
                    e[j]=e[k];
                    e[k]=temp;
                }
            }
        }

        for(int i=0;i<n;i++)
            parent[i]=i;

        printf("\nMST:\n");

        for(int i=0,edges=0;i<m && edges<n-1;i++){

            int u=find(e[i].u);
            int v=find(e[i].v);
```



```

    if(u!=v){

        printf("%d %d %d\n",e[i].u,e[i].v,e[i].w);

        cost+=e[i].w;
        edges++;

        parent[u]=v;
    }
}

printf("\nCost = %d",cost);

return 0;
}

```

OUTPUT

```

Enter number of Vertices and Edges: 4 5
Enter vertex u v and weight w:
0 1 10
0 2 6
0 3 5
1 3 15
2 3 4

MST:
2 3 4
0 3 5
0 1 10

Cost = 19

```

Lab program 9:

Implement Fractional Knapsack using Greedy technique.

SOURCE CODE

```
#include <stdio.h>

int main(){
    int n,rem_weight;
    float total_profit=0;

    printf("\nEnter number of item: ");
    scanf("%d",&n);

    int obj[n],p[n],w[n];

    printf("\nEnter objects: ");
    for(int i=0;i<n;i++)
        scanf("%d",&obj[i]);

    printf("\nEnter profits: ");
    for(int i=0;i<n;i++)
        scanf("%d",&p[i]);

    printf("\nEnter weights: ");
    for(int i=0;i<n;i++)
        scanf("%d",&w[i]);

    printf("\nEnter remaining weight: ");
    scanf("%d",&rem_weight);

    float pw[n];
    for(int i=0;i<n;i++)
        pw[i]=(float)p[i]/w[i];

    for(int i=0;i<n;i++){
        for(int j=i+1;j<n;j++){
            if(pw[i]<pw[j]){
                float temp=pw[i];pw[i]=pw[j];pw[j]=temp;
                temp=obj[i];obj[i]=obj[j];obj[j]=temp;
                temp=p[i];p[i]=p[j];p[j]=temp;
                temp=w[i];w[i]=w[j];w[j]=temp;
            }
        }
    }
}
```

```

for(int k=0;k<n;k++){
    if((rem_weight-w[k]) >= 0){
        rem_weight-=w[k];
        total_profit+=p[k];
    }
    else{
        total_profit+=((float)rem_weight/w[k])*p[k];
        break;
    }
}
printf("\n\nFractional Knapsack:");
printf("\nObject\tProfit\tWeight");
for(int i=0;i<n;i++)
    printf("\n%d\t%d\t%d\t\n",obj[i],p[i],w[i]);
printf("\n\nTotal Profit is: %f\n",total_profit);
return 0;
}

```

OUTPUT

```

Enter number of item: 5

Enter objects: 1 2 3 4 5

Enter profits: 25 20 15 40 50

Enter weights: 3 2 1 4 5

Enter remaining weight: 6


Fractional Knapsack:
Object  Profit  Weight
3       15     1

2       20     2

4       40     4

5       50     5

1       25     3

Total Profit is: 65.000000

```

Lab program 10:

From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.

SOURCE CODE

```
#include <stdio.h>
#define INF 99999

int main() {

    int n,src;

    printf("Enter number of vertices: ");
    scanf("%d",&n);

    int cost[n][n],dist[n],vis[n];

    printf("Enter adjacency matrix:\n");

    for(int i=0;i<n;i++)
        for(int j=0;j<n;j++)
            scanf("%d",&cost[i][j]);

    printf("Enter source vertex: ");
    scanf("%d",&src);

    for(int i=0;i<n;i++){
        dist[i]=cost[src][i];
        vis[i]=0;
    }

    dist[src]=0;
    vis[src]=1;

    for(int i=1;i<n;i++){

        int min=INF,u=-1;

        for(int j=0;j<n;j++){
            if(!vis[j] && dist[j]<min){
                min=dist[j];
                u=j;
            }

            vis[u]=1;

            for(int v=0;v<n;v++)
                if(!vis[v] && dist[u]+cost[u][v]<dist[v])
                    dist[v]=dist[u]+cost[u][v];
        }
    }
}
```

```
}  
  
printf("\nShortest Distances:\n");  
  
for(int i=0;i<n;i++)  
    printf("%d -> %d = %d\n",src,i,dist[i]);  
  
return 0;  
}
```

OUTPUT

```
Enter number of vertices: 3  
Enter adjacency matrix:  
0 5 2  
5 0 1  
2 1 0  
Enter source vertex: 0
```

```
Shortest Distances:  
0 -> 0 = 0  
0 -> 1 = 3  
0 -> 2 = 2
```

Lab program 10:

Implement “N-Queens Problem” using Backtracking.

SOURCE CODE

```
#include <stdio.h>
#include <stdlib.h>
int x[20], n;

int place(int k, int i){
    for(int j=1; j<k; j++){
        if(x[j]==i || abs(x[j]-i)==abs(j-k))
            return 0;
    }
    return 1;
}

void nqueen(int k){
    for(int i=1; i<=n; i++){
        if(place(k,i)){
            x[k]=i;
            if(k==n){
                printf("\nSolution:\n");
                for(int r=1; r<=n; r++){
                    for(int c=1; c<=n; c++){
                        if(x[r]==c)
                            printf("Q ");
                        else
                            printf(". ");
                    }
                    printf("\n");
                }
                printf("\n");
            }
            else
                nqueen(k+1);
        }
    }
}

int main(){

    printf("Enter value of n: ");
    scanf("%d",&n);

    nqueen(1);

    return 0;
}
```

```
}
```

OUTPUT

```
Enter value of n: 4
```

```
Solution:
```

```
. Q . .  
. . . Q  
Q . . .  
. . Q .
```

```
Solution:
```

```
. . Q .  
Q . . .  
. . . Q  
. Q . .
```

268. Missing Number

```
int missingNumber(int* nums, int numsSize) {  
    int expected = numsSize * (numsSize + 1) / 2;  
    int actual = 0;  
  
    for(int i = 0; i < numsSize; i++) {  
        actual += nums[i];  
    }  
  
    return expected - actual;  
}
```

</> Code | Accepted X

← All Submissions

Accepted 122 / 122 testcases passed

lieshaj submitted at Apr 30, 2026 12:40

Analysis

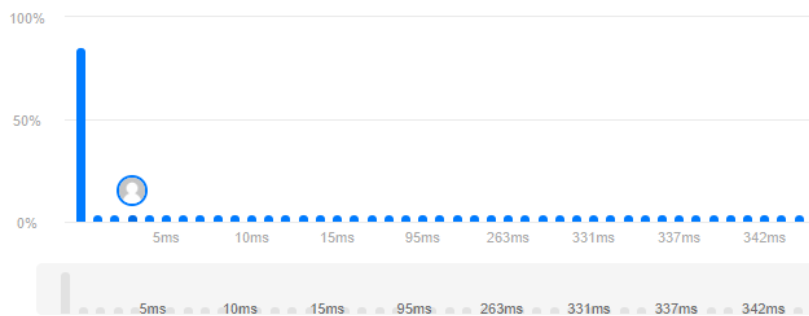
Solution

Runtime

3 ms | Beats 12.81%

Memory

9.58 MB | Beats 79.23%



Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input

nums =
[3,0,1]

Output

2

Expected

1636. Sort Array by Increasing Frequency

```
int freq[201];

int cmp(const void* a, const void* b) {
    int x = *(int*)a;
    int y = *(int*)b;

    if (freq[x + 100] == freq[y + 100])
        return y - x;
    return freq[x + 100] - freq[y + 100];
}

int* frequencySort(int* nums, int numsSize, int* returnSize) {
    *returnSize = numsSize;

    for (int i = 0; i < 201; i++)
        freq[i] = 0;

    for (int i = 0; i < numsSize; i++)
        freq[nums[i] + 100]++;

    qsort(nums, numsSize, sizeof(int), cmp);

    return nums;
}
```

Accepted 180 / 180 testcases passed
lleshaj submitted at Apr 30, 2026 12:41

Analysis Solution

Runtime: 3 ms | Beats: 37.62%
Memory: 11.94 MB | Beats: 53.33%

Code | C

```
1 int freq[201];
```

Testcase Test Result

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input

nums =
[1,1,2,2,2,3]

Output

[3,1,1,2,2,2]

561. Array Partition

```
int compare(const void *a, const void *b) {
    return (*(int *)a - *(int *)b);
}

int arrayPairSum(int* nums, int numsSize) {
    qsort(nums, numsSize, sizeof(int), compare);

    int sum = 0;

    for(int i = 0; i < numsSize; i += 2) {
        sum += nums[i];
    }

    return sum;
}
```

Accepted 85 / 85 testcases passed
lieshaj submitted at Jun 04, 2026 12:53

Analysis

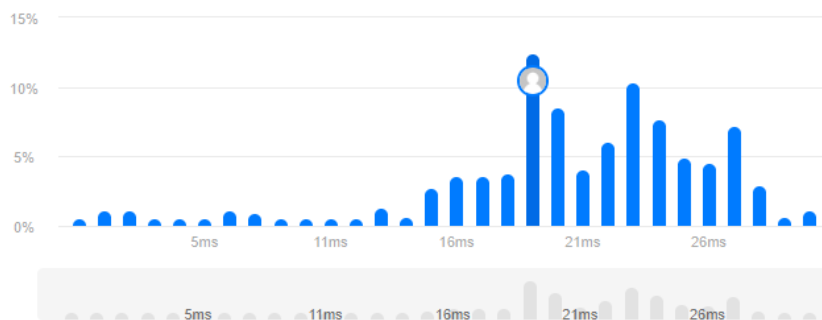
Solution

Runtime

19 ms | Beats 77.20%

Memory

11.60 MB | Beats 33.63%



Testcase | Test Result

Accepted Runtime: 0 ms

Case 1

Case 2

Input

nums =
[1,4,3,2]

Output

4

Expected

3742. Maximum Path Score in a Grid

```
int max(int a, int b) {
    return a > b ? a : b;
}

int maxPathScore(int** grid, int gridSize, int* gridColSize, int k) {
    int m = gridSize;
    int n = gridColSize[0];

    int dp[m][n][k + 1];

    for (int i = 0; i < m; i++)
        for (int j = 0; j < n; j++)
            for (int c = 0; c <= k; c++)
                dp[i][j][c] = -1;

    dp[0][0][0] = 0;

    for (int i = 0; i < m; i++) {
        for (int j = 0; j < n; j++) {
            for (int c = 0; c <= k; c++) {
                if (dp[i][j][c] == -1) continue;

                if (i + 1 < m) {
                    int cost = (grid[i + 1][j] > 0);
                    if (c + cost <= k)
                        dp[i + 1][j][c + cost] =
                            max(dp[i + 1][j][c + cost],
                                dp[i][j][c] + grid[i + 1][j]);
                }

                if (j + 1 < n) {
                    int cost = (grid[i][j + 1] > 0);
                    if (c + cost <= k)
                        dp[i][j + 1][c + cost] =
                            max(dp[i][j + 1][c + cost],
                                dp[i][j][c] + grid[i][j + 1]);
                }
            }
        }
    }

    int ans = -1;
    for (int c = 0; c <= k; c++)
        ans = max(ans, dp[m - 1][n - 1][c]);

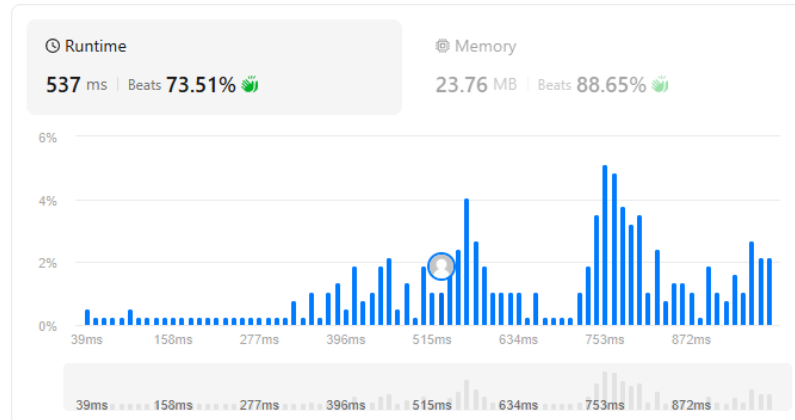
    return ans;
}
```

Accepted 529 / 529 testcases passed

lieshaj submitted at Jun 04, 2026 12:57

Analysis

Solution



Testcase Test Result

Accepted Runtime: 0 ms

Case 1

Case 2

Input

grid =
[[0, 1], [2, 0]]

k =
1

Output

2

LeetCode 70 - Climbing Stairs (C)

```
int climbStairs(int n) {  
    if (n <= 2)  
        return n;  
  
    int a = 1, b = 2, c;  
  
    for (int i = 3; i <= n; i++) {  
        c = a + b;  
        a = b;  
        b = c;  
    }  
  
    return b;  
}
```

</> Code | Accepted ×

← All Submissions →

Accepted 45 / 45 testcases passed

 lieshaj submitted at Jun 04, 2026 13:00

Analysis

Solution

Runtime

0 ms | Beats 100.00% 🏆

Memory

8.60 MB | Beats 36.83%



Testcase | Test Result

Accepted Runtime: 0 ms

Case 1

Case 2

Input

n =
2

Output

2

Expected

1557. Minimum Number of Vertices to Reach All Nodes

```
int* findSmallestSetOfVertices(int n, int** edges, int edgesSize,
                              int* edgesColSize, int* returnSize) {

    int *indegree = (int *)calloc(n, sizeof(int));

    for (int i = 0; i < edgesSize; i++) {
        indegree[edges[i][1]]++;
    }

    int *result = (int *)malloc(n * sizeof(int));
    int k = 0;

    for (int i = 0; i < n; i++) {
        if (indegree[i] == 0) {
            result[k++] = i;
        }
    }

    *returnSize = k;
    free(indegree);

    return result;
}
```

</> Code | Accepted X

← All Submissions

Accepted 66 / 66 testcases passed

lieshaj submitted at Jun 04, 2026 13:02

Analysis

Solution

Runtime

4 ms | Beats 34.48%

Memory

44.43 MB | Beats 27.59%



Runtime	Percentage
2ms	~5%
3ms	~15%
4ms	~45%
6ms	~5%

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 Case 2

Input

n =

6

edges =

[[0,1], [0,2], [2,5], [3,4], [4,2]]

Output

0 2

